Software Testing Document

for

CMD TypeSwift Typing Tutor

Prepared by

Athul Babu K – NIE22CS019

Sobin Seby – NIE22CS047

Sojo Jenson – NIE22CS048

DATE: 02-03-25

Testing Document

1. Introduction

This document outlines the testing strategy for **CMD TypeSwift**, a terminal-based typing tutor application designed to help users improve their typing speed and accuracy, particularly for programming-related content.

2. Test Objectives

1. Verify all functional requirements (user management, lesson handling, typing tests).
2. Ensure non-functional aspects (performance, usability, security).
3. Validate error handling and edge cases.
4. Confirm platform compatibility (Linux terminal environments).

3. Test Scope

In Scope:

- User authentication (login, logout, user creation).
- Lesson management (built-in/custom lessons, content display).
- Typing test functionality (real-time feedback, stats calculation).
- Admin features (user/lesson management).
- Data persistence (user stats, custom lessons).

Out of Scope:

- Network-based features (application is purely local).
- GUI testing (terminal-based only).
- Hardware-specific tests (keyboard input is assumed functional).

4. Test Environment

- **OS:** Linux (tested on Ubuntu 24.04).
- **Tools:**
    - Manual Testing: Terminal with ncurses support.
    - Automation: Shell scripts for repetitive test cases.
    - Static Analysis: gcc -Wall, cppcheck.
- **Dependencies:** ncurses library (libncurses-dev on Linux).

5. Test Cases

5.1 User Authentication

Table 1

ID	Test Case	Steps	Expected Result
TC-01	Admin login	1. Launch app → Enter "admin/admin123".	Access to admin menu.
TC-02	Invalid login	1. Enter incorrect credentials.	Error message, no access.
TC-03	New user creation	1. Select "Create User" → Enter details.	User appears in users.dat.
TC-04	Duplicate username	1. Try to create a user with existing username.	Rejection with error message.

5.2 Lesson Handling

Table 2

ID	Test Case	Steps	Expected Result
TC-05	Load built-in lessons	1. Start app → Check lesson menu.	5+ lessons display correctly.
TC-06	Add custom lesson	1. As admin → "Add Custom Lesson" → Enter title/content.	Lesson appears in menu.
TC-07	Custom lesson from myown.txt	1. Create myown.txt → Run "Custom Lesson".	Content loads without errors.

5.3 Typing Test Functionality

Table 3

ID	Test Case	Steps	Expected Result
TC-08	Run a lesson	1. Select a lesson → Type text.	Real-time colour feedback (G/R).
TC-09	Cancel mid-lesson	1. Press ESC during typing.	Returns to menu, no stats saved.
TC-10	Stats calculation	1. Complete a lesson.	WPM/accuracy displayed correctly.

5.4 Admin Features

Table 4

ID	Test Case	Steps	Expected Result
TC-11	Delete user	1. As admin → Delete a test user.	User removed from users.dat.
TC-12	Delete lesson	1. As admin → Delete a custom lesson.	Lesson file removed from /lessons.

5.5 Edge Cases

Table 5

ID	Test Case	Steps	Expected Result
TC-13	Empty myown.txt	1. Run custom lesson with empty file.	Error message displayed.
TC-14	Max users reached	1. Create users until MAX_USERS (10).	Rejects new users with warning.

6. Test Data

- **User Credentials:**
    - Admin: admin/admin123
    - Test User: testuser/testpass
- **Lesson Files:**
    - Built-in: from main (C Programming, Sample used).
    - Custom: myown.txt (User-provided content).

7. Test Execution

Manual Testing Steps:

1. **Setup:**

gcc -o typeswift typeswift.c -lncurses

./typeswift

2. **Run Test Cases:** Follow steps in Section 5.
3. **Verify:**
    - Check terminal output matches expectations.
    - Inspect generated files (users.dat, /lessons).

Automated Checks:

- Static analysis:

cppcheck --enable=all typeswift.c

- Compilation warnings:

gcc -Wall -Wextra -o typeswift typeswift.c -lncurses

8. Defect Reporting

Table 6

ID	Description	Severity	Status
DEF-01	Crash on invalid lesson index	High	Resolved
DEF-02	Password visible during creation	Medium	Resolved
DEF-03	Inability to choose double digit option	High	Resolved
DEF-04	Lesson number not shown in view lessons after adding a new lesson	High	Resolved

9. Exit Criteria

- All critical test cases (TC-01 to TC-10) pass.
- No high-severity defects unresolved.
- 90%+ code coverage.